

# Compte-rendu Transfert de fichier FTP

Esmeralda Magnin

Matthieu Laurent

28/03/2025

## 1. Introduction

L'objectif de ce mini-projet est de développer un serveur FTP simplifié, permettant à un client de demander un fichier au serveur, qui le lui enverra s'il existe. Cette première étape met en place les fonctionnalités de base pour la transmission de fichiers en utilisant une communication via sockets. Nous avons fait jusqu'à la question 9, la 10 n'a pas pu être faite car comme dit plus loin nous ne savons pas comment gérer le SIGPIPE et notre handler ne le détecte pas non plus.

## 2. Architecture

Le projet est séparé en différentes parties. La partie client avec le fichier FTP\_client.c, la partie serveur avec le fichier FTP\_server.c et un fichier FTP.c contenant les différentes instructions pour le protocole FTP. Un header FTP.h contient des constantes comme le numéro de port, le nombre de fils dans le pool de processus, la structure de données requete\_t pour les requêtes, avec le nom du fichier et le type de commande (GET ou bye).

## 3. Implémentation

### Squelette du Code

L'implémentation est basée sur le code d'un serveur client-serveur classique, adapté aux besoins du projet FTP.

- **Le serveur :**
  - écoute sur le port **1212** (changé par rapport à l'énoncé pour ne pas avoir de problème si exécution sur une machine de l'université).
  - utilise un **pool de processus** pour gérer plusieurs clients simultanément
  - terminaison propre pour tous les processus lors d'une réception du signal **SIGINT**.
- **Le client :**
  - envoie une requête **GET** au serveur, suivi du nombre de caractères dans le nom du fichier et le nom du fichier.
  - reçoit le fichier

## Gestion des Fichiers

- **Côté serveur** : Le serveur conserve ses fichiers depuis un répertoire local **serverfolder/**.
- **Côté client** : Le client enregistre les fichiers dans un répertoire **clientfolder/**.

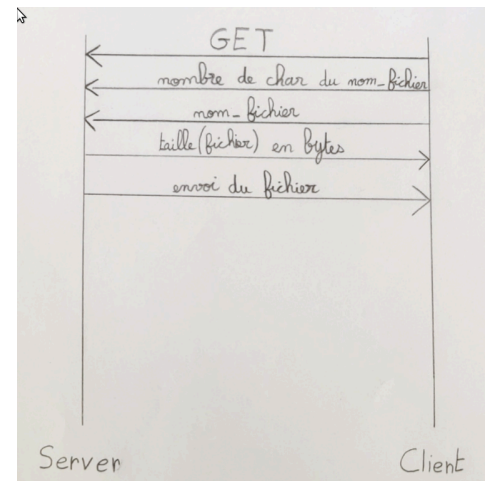
## Transmission des Données

- Le serveur charge le fichier demandé en mémoire.
- Si le fichier est introuvable, il renvoie un code d'erreur.
- Il envoie la taille du fichier, puis le fichier en un seul bloc

## Traitement côté client

Envoi en 1 seul bloc :

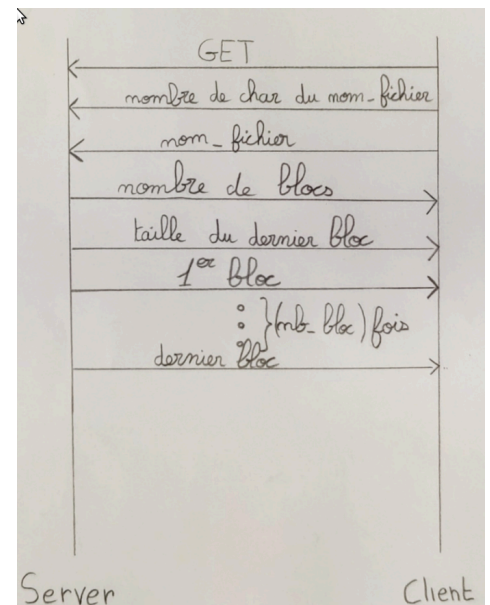
- Le client envoie la commande, puis la taille du nom du fichier (le nombre de caractères) et ensuite le nom du fichier.
- Il reçoit ensuite la taille du fichier, puis le contenu du fichier.
- Il crée un fichier dans le répertoire clientfolder/ et écrit le contenu du fichier dans ce nouveau fichier.



Envoi en plusieurs blocs :

- Le client envoie la commande, puis la taille du nom du fichier (le nombre de caractères) et ensuite le nom du fichier.
- Il crée un fichier dans le répertoire clientfolder/
- Il reçoit ensuite le nombre de blocs d'une taille fixée, puis la taille du dernier bloc, puis le contenu de ce fichier bloc par bloc.
- Il écrit le contenu des blocs dans ce nouveau fichier.

Le client peut demander autant de fichier qu'il veut lors d'une connexion, et peut arrêter sa communication après un échange avec la commande "bye".



## Traitement côté serveur :

Envoi en 1 seul bloc :

- Le serveur attend la réception d'une commande, analyse sa nature.
- Si la commande est "bye", alors il stoppe la connexion avec le client. Si la commande est "GET", alors il se prépare à réceptionner la taille du nom du fichier puis le nom du fichier à transmettre.
- Le serveur vérifie que le fichier est présent dans son dossier serverfolder/, se prépare à l'envoyer s'il est présent, ou infirme le client s'il n'y est pas.
- Le serveur calcule la taille du fichier, puis l'envoi, suivi du contenu du fichier.

Envoi en plusieurs blocs :

- Le serveur attend la réception d'une commande, analyse sa nature.
- Si la commande est "bye", alors il stoppe la connexion avec le client. Si la commande est "GET", alors il se prépare à réceptionner la taille du nom du fichier puis le nom du fichier à transmettre.
- Le serveur vérifie que le fichier est présent dans son dossier serverfolder/, se prépare à l'envoyer s'il est présent, ou infirme le client s'il n'y est pas.
- Le serveur calcule la taille du fichier, puis détermine le nombre de blocs nécessaire pour envoyer la totalité du contenu du fichier. Il calcule aussi la taille du dernier bloc, qui ne sera pas forcément plein.
- Le serveur envoie le nombre de blocs ainsi que la taille du dernier bloc au client, puis commence à envoyer les blocs un par un.

Si le client décide de donner une commande qui n'existe pas, le serveur va lui redemander la commande.

## Gestion des pannes

A chaque envoi ou réception de données de la part du serveur ou du client, il y a une vérification faite pour assurer la bonne communication entre les deux entités. S'il y a un problème d'un côté ou de l'autre, la communication s'arrêtera. Nous n'avons pas su comment gérer SIGPIPE, lorsque le client crash.

## Conclusion

Au cours de ce mini projet, nous avons eu l'occasion d'implémenter progressivement les fonctionnalités d'un serveur de fichier inspiré du protocole FTP, en commençant par une implémentation basique permettant à un client de récupérer un fichier via une requête GET. Ensuite, nous avons amélioré notre serveur en le rendant capable de gérer plusieurs connexions simultanées grâce à un pool de processus. Nous avons également optimisé la transmission d'un fichier en le coupant en bloc, ce qui permet de mieux gérer les fichiers volumineux.